

LAPORAN PRAKTIKUM ALGORITMA DAN STRUKTUR DATA

Function Dan Array



Nama :Muhammad Alfarisi

NIM :2025573010051

Kelas :TI /1C

PROGRAM STUDI TEKNIK INFORMATIKA JURUSAN
INFORMASI DAN KOMPUTER POLITEKNIK NEGERI
LHOKSEUMAWE

2026

DAFTAR ISI

LAPORAN PRAKTIKUM ALGORITMA DAN STRUKTUR DATA Function Dan Array	1
DAFTAR ISI	2
BAB I PENDAHULUAN	3
1.1. Latar Belakang	3
1.2. Identifikasi Masalah	3
1.3. Rumusan Masalah	3
BAB II LANDASAN TEORI	4
2.1. Pengertian Function	4
2.2. Pengertian Array	4
2.3. Fungsion Dasar	4
2.4. Fungsion Arrow dan crowback	4
2.5. Array Dasar dan Array Method	4
2.6. Rekursi	4
BAB III PROSES PRAKTIKUM	5
3.1. Alat Dan Bahan	5
3.2. Proses Praktikum	5
BAB IV PENUTUP	31
4.1. kesimpulan	31

BAB I

PENDAHULUAN

1.1. Latar Belakang

Dalam peningkatan perkembangan yang semakin maju di bidang teknologi khususnya di bidang komputer, banyak programmer yang mulai membuat bahasa pemrograman yang lebih mudah untuk dikuasai dan lebih praktis dalam segi penggunaan nya. Salah satunya ialah JavaScript. sebagai salah satu bahasa pemrograman paling populer yang dijalankan pada runtime Node.js, menawarkan berbagai fitur untuk menangani kompleksitas logika program. Dua konsep fundamental yang sering digunakan adalah Function dan Array. Function memungkinkan pengembang untuk membungkus kode agar dapat digunakan kembali (reusable), sementara Array menyediakan cara yang efektif untuk menyimpan dan mengelola sekumpulan data dalam satu variabel tunggal. Memahami interaksi antara keduanya sangat penting untuk membangun aplikasi yang skalabel dan mudah dipelihara.

1.2. Identifikasi Masalah

1. Kesulitan dalam mengorganisir kode yang berulang tanpa menggunakan fungsi.
2. Kurangnya pemahaman dalam mengelola kumpulan data yang dinamis menggunakan array.
3. Kompleksitas penggunaan metode-metode modern pada JavaScript seperti arrow functions dan callback.
4. Tantangan dalam memecahkan masalah logika yang bersifat repetitif atau bertingkat menggunakan teknik rekursi.

1.3. Rumusan Masalah

- Apa pengertian dan kegunaan dari Function dan Array dalam JavaScript?
- Bagaimana cara mengimplementasikan Basic Function, Arrow Function, dan Callback pada Node.js?
- Bagaimana cara memanipulasi data di dalam Array menggunakan berbagai metode bawaan JavaScript?
- Bagaimana konsep rekursi diterapkan dalam menyelesaikan persoalan pemrograman secara efektif?

BAB II

LANDASAN TEORI

2.1. Pengertian Function

Function atau fungsi adalah blok kode yang dirancang untuk melakukan tugas tertentu. Fungsi hanya akan dieksekusi ketika "dipanggil" atau diinvokasi. Dengan fungsi, kode menjadi lebih modular, mudah dibaca, dan meminimalisir redundansi.

2.2. Pengertian Array

Array adalah objek khusus dalam JavaScript yang digunakan untuk menyimpan daftar elemen. Berbeda dengan variabel biasa yang hanya menyimpan satu nilai, satu variabel array dapat menampung berbagai tipe data (seperti string, number, bahkan object) dalam indeks yang terurut dimulai dari 0.

2.3. Fungsi Dasar

Deklarasi fungsi dasar biasanya menggunakan kata kunci `function` diikuti dengan nama fungsi, parameter dalam kurung, dan blok kode. Contoh: `function sapa(nama) { return "Halo " + nama; }`.

2.4. Fungsi Arrow dan Callback

Arrow Function adalah sintaks penulisan fungsi yang lebih ringkas menggunakan simbol `=>`. Sementara itu, Callback adalah fungsi yang dilewatkan sebagai argumen ke dalam fungsi lain untuk dieksekusi kemudian, yang sangat krusial dalam pemrograman asinkronus di Node.js.

2.5. Array Dasar dan Array Method

Array dasar didefinisikan menggunakan kurung siku `[]`. Untuk memanipulasinya, JavaScript menyediakan method bawaan seperti `.push()` (tambah elemen), `.pop()` (hapus elemen terakhir), `.map()` (transformasi data), dan `.filter()` (menyaring data).

2.6. Rekursi

Rekursi adalah teknik di mana sebuah fungsi memanggil dirinya sendiri untuk menyelesaikan sub-masalah yang lebih kecil. Setiap fungsi rekursif harus memiliki base case (kondisi berhenti) untuk mencegah terjadinya infinite loop atau stack overflow.

BAB III

PROSES PRAKTIKUM

3.1. Alat Dan Bahan

- Laptop
- Virtual Studio Code
- Git
- GitHub
- Node JS.

3.2. Proses Praktikum

Langkah 1 Membuat folder dalam repository dan masukan ke repository sebelumnya.

```
Mkdir praktikum-3  
Cd praktikum-3
```

Langkah 2 Inisialisasi npm project

```
npm init -y
```

Langkah 3 Buat file bernama 01-function-dasar.js

```
Touch 01-function-dasar.js
```

Function dan array

Langkah 4 Isi file 01-function-dasar.js dengan code berikut

```
// Membuat function untuk dasar.js

// mendeklarasikan function

//=====

// --- Membuat function ---
function sapa() {
    console.log("Halo! Selamat datang di praktikum JavaScript.");
}
sapa(); //berfungsi untuk memanggil fungsi sapa
sapa(); //bisa untuk memanggil fungsi sapa berulang kali
//=====

// --- Membuat parameter untuk function ---

function sapaNama(nama) {
    console.log(`Halo, ${nama}! Selamat belajar.`);
}
sapaNama("Budi"); //berfungsi untuk memanggil fungsi sapa dengan parameter 'Budi'
sapaNama("Siti"); //berfungsi untuk memanggil fungsi sapa dengan parameter 'Siti'
//=====

// --- Membuat return value dan beberapa parameter untuk function ---
function tambah(angka1, angka2) {
    const hasil = angka1 + angka2;
    return hasil; //berfungsi untuk mengambil nilai hasil penjumlahan
}

const hasilpenjumlahan = tambah(10, 25);
console.log("10 + 25 =", hasilpenjumlahan);
```

Function dan array

```
console.log("7 + 13 =", tambah(7, 13)); //untuk langsung mencetak
hasil penjumlahan
//=====================================================

// --- membuat function dengan default parameter ---
// --- jika tidak mendapatkan argumen, maka gunakan nilai default ---
function hitung(nilai, pengali = 2) {
    return nilai * pengali;
}
console.log(hitung(5)); // Output: 10 (pengali menggunakan nilai
default: 2)
console.log(hitung(5, 3)); // Output: 15 (pengali yang diisi: 3
digunakan)
//=====================================================

// --- membuat scope variabel ---
const pesanGlobal = "saya ada di mana saja"; //scope global

function cekScope() {
    const pesanLokal = "saya hanya ada di dalam fungsi ini"; //scope
    lokal
    console.log(pesanGlobal); // bisa diakses melalui global
    console.log(pesanLokal); // bisa diakses melalui lokal
}

cekScope();
console.log(pesanGlobal); // bisa diakses melalui global
//=====================================================

// --- latihan 1: ---
// --- membuat function perkurangan ---
function kurang(a, b) {
    const hasil = a - b;
    return hasil;
}
const hasilpengurangan = kurang(20, 5);
console.log("20 - 5 =", hasilpengurangan);
console.log("15 - 7 =", kurang(15, 7));
```

Function dan array

```
// --- membuat function perkalian ---
function kali(a, b) {
  const hasil = a * b;
  return hasil;
}
const hasilperkalian = kali(6, 4);
console.log("6 * 4 =", hasilperkalian);
console.log("8 * 3 =", kali(8, 3));

// --- membuat function pembagian ---
function bagi(a, b) {
  const hasil = a / b;
  if (b === 0) {
    return "Error: Pembagian dengan nol tidak diperbolehkan.";
    return null;
  }
  return hasil;
}
const hasilpembagian = bagi(10, 2);
console.log("10 / 2 =", hasilpembagian);
console.log("15 / 3 =", bagi(15, 3));
//=====

// --- membuat function kalkulator sederhana menggunakan if/else if -
--
function kalkulator(angka1, angka2, operator) {
  if (operator === "+") {
    return angka1 + angka2;
  } else if (operator === "-") {
    return angka1 - angka2;
  } else if (operator === "*") {
    return angka1 * angka2;
  } else if (operator === "/") {
    if (angka2 === 0) {
      return "Error: Pembagian dengan nol tidak diperbolehkan.";
    }
    return angka1 / angka2;
  }
}
```

Function dan array

```
    } else {  
        return "Error: Operator tidak valid.";  
    }  
}  
console.log("\n=== Kalkulator dengan Operator ===");  
function kalkulator(angka1, angka2, operator) {  
    switch (operator) {  
        case "+":  
            return tambah(angka1, angka2);  
        case "-":  
            return kurang(angka1, angka2);  
        case "*":  
            return kali(angka1, angka2);  
        case "/":  
            if (angka2 === 0) {  
                return "Error: Pembagian dengan nol tidak diperbolehkan.";  
            }  
            return bagi(angka1, angka2);  
        default:  
            return "Error: Operator tidak valid.";  
    }  
}  
  
console.log("\n 5 + 3 =", kalkulator(5, 3, "+"));  
console.log("\n 10 - 4 =", kalkulator(10, 4, "-"));  
console.log("\n 6 * 7 =", kalkulator(6, 7, "*"));  
console.log("\n 9 / 3 =", kalkulator(9, 3, "/"));  
console.log("\n 7 / 0 =", kalkulator(7, 0, "/"));  
  
console.log(kalkulator(10, 5, "+"));
```

Langkah 5 Simpan lalu jalankan pada tampilan terminal

Node 01-function-dasar.js

Function dan array

Hasil Run terminal untuk 01-function-dasar.js

```
ASUS@LAPTOP-6BI2HB8D MINGW64 /d/2025573010051_Algoritma-Dan-Struktur-Data (main)
$ cd praktikum-3

ASUS@LAPTOP-6BI2HB8D MINGW64 /d/2025573010051_Algoritma-Dan-Struktur-Data/praktikum-3 (main)
$ node 01-function-dasar.js
Halo, selamat datang di praktikum JavaScript!

Halo, selamat datang di praktikum JavaScript!

Halo, Andi! Selamat datang di praktikum JavaScript!

Halo, Randi! Selamat datang di praktikum JavaScript!

10 + 5 = 15
20 + 15 = 35
10 - 7 = 3
10 - 6 = 4

Nama Global: Global
Nama Local: Local

=== Function Kalkulator===

5 + 3 = 8

10 - 4 = 6

6 * 7 = 42

9 / 3 = 3
```

Langkah 6 Buat file dengan nama 02-arrow-callback.js

Function dan array

Touch 02-arrow-callback.js

Langkah 7 Isi file 02-arrow-callback.js dengan code berikut

```
// 02-arrow-callback.js
//=====

// --- Membuat function dengan arrow ---
// --- function cara lama, menggunakan declaration ---
function kuadratBiasa (x) {
    return x * x;
}

// function cara baru, menggunakan arrow function
const kuadratArrow = (x) => {
    return x * x;
};

// cara ringkas function arrow mengimPLICIT return
const kuadratRingkas = x => x * x;

console.log('=== Perbandingan Penulisan ===');
console.log('Biasa :', kuadratBiasa(5));
console.log('Arrow :', kuadratArrow(5));
console.log('Ringkas :', kuadratRingkas(5));

// --- membuat arrow function dengan beberapa parameter ---
const luas = (panjang, lebar) => panjang * lebar;
const salam = (nama, waktu) => `Selamat ${waktu}, ${nama}!`;
console.log('Luas 4x6 :', luas(4, 6));
console.log(salam('Budi', 'Pagi'));

// --- memanggil kembali function dengan callback sebagai argumen --
-
function lakukanOperasi(angka, operasiCallback) {
```

Function dan array

```
    console.log(`Angka awal: ${angka}`);
    const hasil = operasiCallback(angka);
    console.log(`Hasil setelah operasi: ${hasil}`);
}

console.log('\n=== Callback ===');
lakukanOperasi(7, kuadratRingkas); // mengirimkan function
sebagai argument
lakukanOperasi(10, x => x + 100); // mengirim arrow function
secara langsung
lakukanOperasi(20, function(x) { // mengirim function biasa
secara langsung
    return x / 2;
});

// --- setTimeout untuk contoh penggunaan callback ---
console.log('\n=== setTimeout dengan Callback ===');
console.log('pesan 1: sebelum timer');

setTimeout(() => {
    console.log('pesan 3: ini dari dalam setTimeout (setelah 1
detik)');
}, 1000); //mili detik

    console.log('pesan 2: setelah mendaftarkan timer');

// LATIHAN 2 PIPELINE TRANSFORMASI
const keHurufBesar = (str) => str.toUpperCase();
const tambahTandaSeru = (str) => str + "!";
const hitungKata = (str) => str.split(" ").length;

function prosesTeks(teks, transformasiCallback) {
    return transformasiCallback(teks);
}

console.log("\n=== Pipeline Transformasi ===");
const teksAwal = "Belajar JavaScript itu menyenangkan";
const hasilTransformasi = prosesTeks(teksAwal, (teks) => {
    const besar = keHurufBesar(teks);
```

Function dan array

```
const seru = tambahTandaSeru(besar);
const jumlahKata = hitungKata(seru);
console.log(`Teks awal: ${teks}`);
return `Besar: ${besar}, Transformasi: ${seru} (Jumlah kata:
${jumlahKata})`;
});
console.log(hasilTransformasi);
```

Langkah 8 Simpan lalu jalankan pada tampilan terminal

Node 02-arrow-callback.js

Hasil Run terminal untuk 02-arrow-callback.js

```
ASUS@LAPTOP-6BI2HB8D MINGW64 /d/2025573010051_Algoritma-Dan-Struktur-Data/praktikum-3 (main)
$ node 02-arrow-callback.js
=== Perbandingan Penulisan ===
Biasa : 25
Arrow : 25
Ringkas : 25
Luas 4x6 : 24
Selamat Pagi, Budi!

=== Callback ===
Angka awal: 7
Hasil setelah operasi: 49
Angka awal: 10
Hasil setelah operasi: 110
Angka awal: 20
Hasil setelah operasi: 10

=== setTimeout dengan Callback ===
pesan 1: sebelum timer
pesan 2: setelah mendaftarkan timer

=== Pipeline Transformasi ===
Teks awal: Belajar JavaScript itu menyenangkan
Besar: BELAJAR JAVASCRIPT ITU MENYENANGKAN, Transformasi: BELAJAR JAVASCRIPT ITU MENYENANGKAN! (Jumlah kata: 4)
pesan 3: ini dari dalam setTimeout (setelah 1 detik)
```

Langkah 9 Membuat file 03-array-dasar.js

Function dan array

Touch 03-array-dasar.js

Langkah 10 Isi file 03-array-dasar.js dengan code berikut

```
// --- 03-array-dasar.js
//=====================================================
// membuat array : deklarasi, akses, dan manipulasi

// pembuatan array ---
const mahasiswa = ["Budi", "Siti", "Ahmad", "Rina"];
const nilai = [85, 92, 95, 88];
console.log("=== Array Awal ===");
console.log("Mahasiswa:", mahasiswa);
console.log("Nilai :", nilai);
console.log("Jumlah mahasiswa:", mahasiswa.length);

// --- akses elemen array ---
console.log("\n=== Akses Elemen ===");
console.log("Elemen pertama :", mahasiswa[0]); // memulai indeks 0
console.log("Elemen ketiga :", mahasiswa[2]); // memulai indeks 2
console.log("Elemen terakhir:", mahasiswa[mahasiswa.length - 1]);

// --- merubah element ---
mahasiswa[1] = "Siti Rahayu"; // merubah elemen ke indeks 1
console.log("\nSetelah diubah:", mahasiswa);

// --- Menambahkan dan juga menghapuskan elemen ---
console.log("\n=== Manipulasi Array ===");
mahasiswa.push("Doni"); // menambahkan elemen di akhir array
console.log("Setelah push :", mahasiswa);

mahasiswa.unshift("Zahra"); // menambah di awal
console.log("Setelah unshift :", mahasiswa);
```

Function dan array

```
const dihapusAkhir = mahasiswa.pop(); // menghapus dari akhir
console.log("Dihapus (pop) :", dihapusAkhir);
console.log("Setelah pop :", mahasiswa);

const dihapusAwal = mahasiswa.shift(); // menghapus dari awal
console.log("Dihapus (shift) :", dihapusAwal);
console.log("Setelah shift :", mahasiswa);

// --- mencari elemen dalam array ---
console.log("\n=== Pencarian ===");
console.log("Indeks Ahmad :", mahasiswa.indexOf("Ahmad"));
console.log("Ada Rina? :", mahasiswa.includes("Rina"));
console.log("Ada Budi? :", mahasiswa.includes("Budi"));

// --- mengiris array dengan slice ---
const sebagian = nilai.slice(1, 4); // dari indeks 1 sampai sebelum 4
console.log("\nNilai asli :", nilai);
console.log("Slice [1,4] :", sebagian);

// LATIHAN MANAJEMEN DAFTAR BELANJA
const daftarBelanja = ["Susu", "Roti", "Telur"];
console.log("\n=== Daftar Belanja Awal ===");
for (let i = 0; i < daftarBelanja.length; i++) {
    console.log(daftarBelanja[i]);
}

daftarBelanja.push("Beras");
daftarBelanja.push("Gula");
console.log("\n=== Daftar Belanja Baru ===");
for (let i = 0; i < daftarBelanja.length; i++) {
    console.log(daftarBelanja[i]);
}

daftarBelanja.shift(0);

console.log("\n=== Daftar Belanja Terbaru ===");
for (let i = 0; i < daftarBelanja.length; i++) {
    console.log(daftarBelanja[i]);
}
```

Function dan array

```
}
```

```
console.log(`Adakah susu? ${daftarBelanja.includes("Susu")}`);
```

Langkah 11 Simpan dan jalankan pada terminal

Node 03-array-dasar.js

Hasil Run terminal untuk 03-array-dasar.js

Function dan array

```

ASUS@LAPTOP-6BI2HB8D MINGW64 /d/2025573010051_Algoritma-Dan-Struktur-Data/praktikum-3 (main)
$ node 03-array-dasar.js
=== Array Awal ===
Mahasiswa: [ 'Budi', 'Siti', 'Ahmad', 'Rina' ]
Nilai : [ 85, 92, 95, 88 ]
Jumlah mahasiswa: 4

=== Akses Elemen ===
Elemen pertama : Budi
Elemen ketiga : Ahmad
Elemen terakhir: Rina

Setelah diubah: [ 'Budi', 'Siti Rahayu', 'Ahmad', 'Rina' ]

=== Manipulasi Array ===
Setelah push : [ 'Budi', 'Siti Rahayu', 'Ahmad', 'Rina', 'Doni' ]
Setelah unshift : [ 'Zahra', 'Budi', 'Siti Rahayu', 'Ahmad', 'Rina', 'Doni' ]
Dihapus (pop) : Doni
Setelah pop : [ 'Zahra', 'Budi', 'Siti Rahayu', 'Ahmad', 'Rina' ]
Dihapus (shift) : Zahra
Setelah shift : [ 'Budi', 'Siti Rahayu', 'Ahmad', 'Rina' ]

=== Pencarian ===
Indeks Ahmad : 2
Ada Rina? : true
Ada Budi? : true

Nilai asli : [ 85, 92, 95, 88 ]
Slice [1,4] : [ 92, 95, 88 ]

=== Daftar Belanja Awal ===

```

Langkah 12 Membuat file bernama 04-array-method.js

Touch 04-array-method.js

Langkah 13 isi file 04-array-method.js dengan code berikut

```

// --- 04-array-method.js
//=====
// membuat methode iterasi dengan menggunakan forEach, map, filter,
dan reduce

```

Function dan array

```
//=====

const nilaiMahasiswa = [75, 90, 82, 68, 95, 48, 77];

// --- fungsi forEach untuk menjalankan sesuatu di setiap elemen ---
-
console.log("=== forEach: Tampilkan Semua Nilai ===");
nilaiMahasiswa.forEach((nilai, indeks) => {
  console.log(` Mahasiswa-${indeks + 1}: ${nilai}`);
});

// --- map berfungsi sebagai transformasi setiap menjadi nilai baru ---
--

// cara konversi nilai angka menjadi grade huruf
const gradeHuruf = nilaiMahasiswa.map((nilai) => {
  if (nilai >= 90) return "A";
  if (nilai >= 80) return "B";
  if (nilai >= 70) return "C";
  if (nilai >= 60) return "D";
  return "E";
});
console.log("\n=== map: Nilai ke Grade ===");
console.log("Nilai :", nilaiMahasiswa);
console.log("Grade :", gradeHuruf);

// --- filter berfungsi sebagai penyaringan elemen yang memenuhi kondisi ---
// membuat filter
const nilaiLulus = nilaiMahasiswa.filter((nilai) => nilai >= 60);
const nilaiGagal = nilaiMahasiswa.filter((nilai) => nilai < 60);

console.log("\n=== filter: Lulus dan Tidak Lulus ===");
console.log("Semua nilai :", nilaiMahasiswa);
console.log("Nilai lulus :", nilaiLulus);
console.log("Nilai gagal :", nilaiGagal);

// --- reduce berfungsi mereduksi array menjadi satu nilai ---
```

Function dan array

```
const totalNilai = nilaiMahasiswa.reduce((akumulator, nilai) => {
  return akumulator + nilai;
}, 0); // 0 adalah nilai default akumulator

const rataRata = totalNilai / nilaiMahasiswa.length;

console.log("\n=== reduce: Statistik Nilai ===");
console.log("Total nilai :", totalNilai);
console.log("Rata-rata :", rataRata.toFixed(2));

// ---Menggabungkan beberapa metode menjadi satu (method chaining) ---
const rataRataNilaiLulus = nilaiMahasiswa
  .filter((nilai) => nilai >= 60) // mengambil dari yang lulus
  terlebih dulu
  .reduce((acc, val, idx, arr) => {
    return acc + val / arr.length; // bagi saat proses
  }, 0);

console.log("\n=== Method Chaining ===");
console.log("Rata-rata nilai lulus:", rataRataNilaiLulus.toFixed(2));

// LATIHAN: ANALISIS DATA PRODUK
const produk = [
  { nama: "Laptop", harga: 8500000, stok: 5 },
  { nama: "Mouse", harga: 150000, stok: 0 },
  { nama: "Keyboard", harga: 450000, stok: 12 },
  { nama: "Monitor", harga: 3200000, stok: 0 },
  { nama: "Headset", harga: 350000, stok: 8 },
];

console.log("\n\n=== Produk Tersedia ===");
produk
  .filter((filterProduk) => filterProduk.stok > 0)
  .forEach((produk) => {
    console.log(
      `Nama: ${produk.nama}, Harga: ${produk.harga}, Stok:
      ${produk.stok}`,
    );
  });
```

Function dan array

```
    );  
  });  
  
console.log("\n=== Status Produk ===");  
produk  
  .map((mapProduk) => {  
    return {  
      nama: mapProduk.nama,  
    };  
  })  
  .forEach((produk) => {  
    console.log(produk);  
  });  
  
console.log("\n=== Total Nilai Stok ===");  
const totalNilaiStok = produk.reduce((total, produk) => {  
  return total + produk.harga * produk.stok;  
}, 0);  
console.log(`Total nilai stok: ${totalNilaiStok}`);  
  
produk.forEach((produk) => {  
  if (produk.stok > 0) {  
    console.log(  
      `[TERSEDIA] ${produk.nama} - ${produk.harga} [${produk.stok}  
unit]`,  
    );  
  } else {  
    console.log(`[HABIS] ${produk.nama} - ${produk.harga}`);  
  }  
});
```

Langkah 14 Simpan dan jalankan codenya pada terminal

Node 04-array-method.js

Function dan array

Hasil Run terminal untuk 04-array-method.js

```
ASUS@LAPTOP-6BI2HB8D MINGW64 /d/2025573010051_Algoritma-Dan-Struktur-Data/praktikum-3 (main)
• $ node 04-array-method.js
=== forEach: Tampilkan Semua Nilai ===
Mahasiswa-1: 75
Mahasiswa-2: 90
Mahasiswa-3: 82
Mahasiswa-4: 68
Mahasiswa-5: 95
Mahasiswa-6: 48
Mahasiswa-7: 77

=== map: Nilai ke Grade ===
Nilai : [
  75, 90, 82, 68,
  95, 48, 77
]
Grade : [
  'C', 'A', 'B',
  'D', 'A', 'E',
  'C'
]

=== filter: Lulus dan Tidak Lulus ===
Semua nilai : [
  75, 90, 82, 68,
  95, 48, 77
]
Nilai lulus : [ 75, 90, 82, 68, 95, 77 ]
Nilai gagal : [ 48 ]

=== reduce: Statistik Nilai ===
Total nilai : 535
Rata-rata : 76.43

=== Method Chaining ===
```

Ln 103, Col 1 Spaces: 2 UTF-8 CRLF { } JavaScript  Inline suggestions quota reached (Go Live) Wind

Langkah 15 Buat file bernama 05-rekursi.js

Touch 05-rekursi.js

Function dan array

Langkah 16 Isi file 05-rekursi.js dengan code berikut

```
// --- 1. Faktorial:  $n! = n \times (n-1)!$  ---
function faktorial(n) {
  if (n <= 1) return 1;
  return n * faktorial(n - 1);
}
console.log('=== Faktorial ===');
console.log('0! =', faktorial(0));
console.log('1! =', faktorial(1));
console.log('5! =', faktorial(5));
console.log('7! =', faktorial(7));

// --- 2. Fibonacci:  $F(n) = F(n-1) + F(n-2)$  ---
// Deret: 0, 1, 1, 2, 3, 5, 8, 13, 21, ...
function fibonacci(n) {
  // Base case
  if (n === 0) return 0;
  if (n === 1) return 1;
  // Recursive case
  return fibonacci(n - 1) + fibonacci(n - 2);
}
console.log('\n=== Fibonacci ===');
for (let i = 0; i <= 8; i++) {
  process.stdout.write(fibonacci(i) + ' ');
}
console.log('');

// --- 3. Rekursi untuk menelusuri array ---
function jumlahArray(arr, indeks = 0) {
  // Base case: sudah sampai akhir array
  if (indeks === arr.length) return 0;
  // Recursive case: elemen saat ini + jumlah sisa array
  return arr[indeks] + jumlahArray(arr, indeks + 1);
}
```

Function dan array

```
const angka = [3, 7, 2, 9, 5];
console.log('\n=== Jumlah Array dengan Rekursi ===');
console.log('Array :', angka);
console.log('Jumlah :', jumlahArray(angka));

// --- 4. Countdown rekursif ---
function countdown(n) {
  if (n < 0) {
    console.log('Selesai!');
    return;
  }
  console.log(n);
  countdown(n - 1);
}
console.log('\n=== Countdown Rekursif ===');
countdown(5);
```

Langkah 17 Simpan dan jalankan codenya pada terminal

Node 05-rekursi.js

Function dan array

Hasil Run terminal untuk 05-rekursi.js

```
ASUS@LAPTOP-6BI2HB8D MINGW64 /d/2025573010051_Algoritma-Dan-Struktur-Data/praktikum-3 (main)
● $ node 05-rekursi.js
=== Faktorial ===
0! = 1
1! = 1
5! = 120
7! = 5040

=== Fibonacci ===
0 1 1 2 3 5 8 13 21

=== Jumlah Array dengan Rekursi ===
Array : [ 3, 7, 2, 9, 5 ]
Jumlah : 26

=== Countdown Rekursif ===
5
4
3
2
1
0
Selesai!
Ha
ASUS@LAPTOP-6BI2HB8D MINGW64 /d/2025573010051_Algoritma-Dan-Struktur-Data/praktikum-3 (main)
○ $
```

Langkah 18 Buat folder serta file baru untuk Tugas

```
Mkdir tugas
Touch tugas/tugas-1.js tugas/tugas-2.js
```

Langkah 19 Masuk ke file tugas-1.js dan masukan code berikut

Function dan array

```
console.log("=== Sistem Nilai Mahasiswa ===");

const dataMahasiswa = [
  { nama: "Alex", nilai: 55 },
  { nama: "Bambang", nilai: 70 },
  { nama: "Citra", nilai: 93 },
  { nama: "Sul", nilai: 50 },
  { nama: "Mirza", nilai: 70 },
  { nama: "Febri", nilai: 89 },
];

function hitungStatistik(arrMahasiswa) {
  const hasil = arrMahasiswa.reduce(
    (acc, mhs) => {
      acc.totalNilai += mhs.nilai;

      if (mhs.nilai > acc.tertinggi) {
        acc.tertinggi = mhs.nilai;
      }

      if (mhs.nilai < acc.terendah) {
        acc.terendah = mhs.nilai;
      }

      return acc;
    },
    {
      totalNilai: 0,
      tertinggi: arrMahasiswa[0].nilai,
      terendah: arrMahasiswa[0].nilai,
    },
  );

  return {
    total: arrMahasiswa.length,
    rataRata: (hasil.totalNilai / arrMahasiswa.length).toFixed(2),
    tertinggi: hasil.tertinggi,
    terendah: hasil.terendah,
  };
}
```

Function dan array

```
};  
}  
  
function filterLulus(arrMahasiswa, batasLulus) {  
  return arrMahasiswa.filter((mhs) => mhs.nilai >= batasLulus);  
}  
  
function tambahGrade(arrMahasiswa) {  
  return arrMahasiswa.map((mhs) => {  
    let grade;  
  
    if (mhs.nilai >= 85) {  
      grade = "A";  
    } else if (mhs.nilai >= 75) {  
      grade = "B";  
    } else if (mhs.nilai >= 65) {  
      grade = "C";  
    } else if (mhs.nilai >= 50) {  
      grade = "D";  
    } else {  
      grade = "E";  
    }  
  
    return {  
      ...mhs,  
      grade: grade,  
    };  
  });  
}  
  
// ===== Memanggil Function =====  
  
// Hitung statistik  
const statistik = hitungStatistik(dataMahasiswa);  
  
// Filter mahasiswa lulus (misal batas 70)  
const mahasiswaLulus = filterLulus(dataMahasiswa, 70);
```

Function dan array

```
// Tambah grade
const mahasiswaDenganGrade = tambahGrade(dataMahasiswa);

// ===== Tampilkan Hasil =====

console.log("\n=== Statistik Nilai ===");
console.log(`Total Mahasiswa : ${statistik.total}`);
console.log(`Rata-rata Nilai: ${statistik.rataRata}`);
console.log(`Nilai Tertinggi: ${statistik.tertinggi}`);
console.log(`Nilai Terendah : ${statistik.terendah}`);

console.log("\n=== Mahasiswa Lulus (>= 70) ===");

mahasiswaLulus.forEach((mhs) => {
  console.log(` - ${mhs.nama} (${mhs.nilai})`);
});

console.log("\n=== Daftar Mahasiswa + Grade ===");

mahasiswaDenganGrade.forEach((mhs) => {
  console.log(`${mhs.nama} - Nilai: ${mhs.nilai}, Grade: ${mhs.grade}`);
});
```

Langkah 20 Simpan dan jalankan codenya pada terminal

Node tugas-1.js

Function dan array

Hasil Run terminal untuk tugas-1.js

```
ASUS@LAPTOP-6BI2HB8D MINGW64 /d/2025573010051_Algoritma-Dan-Struktur-Data/praktikum-3/tugas (main)
$ node tugas-1.js
=== Sistem Nilai Mahasiswa ===

=== Statistik Nilai ===
Total Mahasiswa : 6
Rata-rata Nilai: 71.17
Nilai Tertinggi: 93
Nilai Terendah : 50

=== Mahasiswa Lulus (>= 70) ===
- Bambang (70)
- Citra (93)
- Mirza (70)
- Febri (89)

=== Daftar Mahasiswa + Grade ===
Alex - Nilai: 55, Grade: D
Bambang - Nilai: 70, Grade: C
Citra - Nilai: 93, Grade: A
Sul - Nilai: 50, Grade: D
Mirza - Nilai: 70, Grade: C
Febri - Nilai: 89, Grade: A
ASUS@LAPTOP-6BI2HB8D MINGW64 /d/2025573010051_Algoritma-Dan-Struktur-Data/praktikum-3/tugas (main)
$
```

Langkah 21 Masuk ke file tugas-2.js dan isi dengan code berikut

```
console.log("=== Pangkat dan Palindrom Rekursif ===");

function pangkat(basis, eksponen) {
  if (eksponen === 0) {
    return 1;
  }
}
```

Function dan array

```
    return basis * pangkat(basis, eksponen - 1);
}

console.log("\n=== Uji Fungsi Pangkat ===");
console.log(`2 pangkat 3 = ${pangkat(2, 3)}`);
console.log(`5 pangkat 2 = ${pangkat(5, 2)}`);
console.log(`3 pangkat 4 = ${pangkat(3, 4)}`);

function balikString(str) {
    if (str.length <= 1) {
        return str;
    }

    return str[str.length - 1] + balikString(str.slice(0, str.length - 1));
}

console.log("\n=== Uji Fungsi Balik String ===");
console.log(`'halo' dibalik menjadi '${balikString("halo")}'`);
console.log(`'javascript' dibalik menjadi '${balikString("javascript")}'`);

function cekPalindrom(str) {
    const hasilBalik = balikString(str);

    if (str === hasilBalik) {
        return true;
    } else {
        return false;
    }
}

console.log("\n=== Uji Cek Palindrom ===");

const kataUji = ["katak", "civic", "level", "halo", "meja"];

kataUji.forEach((kata) => {
```

Function dan array

```
if (cekPalindrom(kata)) {  
    console.log(`${kata} adalah palindrom`);  
} else {  
    console.log(`${kata} bukan palindrom`);  
}  
});
```

Langkah 22 Simpan dan jalankan pada terminal

Node tugas-2.js

Hasil Run terminal untuk tugas-2.js

```
ASUS@LAPTOP-6BI2HB8D MINGW64 /d/2025573010051_Algoritma-Dan-Struktur-Data/praktikum-3/tugas (main)  
● $ node tugas-2.js  
=== Pangkat dan Palindrom Rekursif ===  
  
=== Uji Fungsi Pangkat ===  
2 pangkat 3 = 8  
5 pangkat 2 = 25  
3 pangkat 4 = 81  
  
=== Uji Fungsi Balik String ===  
'halo' dibalik menjadi 'olah'  
'javascript' dibalik menjadi 'tpircsavaj'  
  
=== Uji Cek Palindrom ===  
katak adalah palindrom  
civic adalah palindrom  
level adalah palindrom  
halo bukan palindrom  
meja bukan palindrom  
ASUS@LAPTOP-6BI2HB8D MINGW64 /d/2025573010051_Algoritma-Dan-Struktur-Data/praktikum-3/tugas (main)  
○ $
```

BAB IV

PENUTUP

4.1. kesimpulan

Berdasarkan hasil praktikum, dapat disimpulkan bahwa Function dan Array adalah komponen esensial dalam JavaScript yang saling melengkapi. Penggunaan function meningkatkan modularitas kode, sementara array mempermudah manajemen koleksi data. Fitur-fitur modern seperti Arrow Functions dan metode Array bawaan sangat membantu mempercepat proses penulisan kode di lingkungan Node.js.